

LSTM: A Search Space Odyssey

LSTM: A Search Space Odyssey

Klaus Greff, Rupesh K. Srivastava, Jan Koutník, Bas R. Steunebrink, Jürgen Schmidhuber
(2017)

익숙한 LSTM 구성요소를 하나씩 바꾸어 실제 필요성을 검증한 대규모 비교 연구

연구 질문을 실험 규모로 바꾸기

익숙한 LSTM 부품 중 무엇이 필요한지 architecture와 task를 교차해 검사한다

축	수	실험에서 뜻하는 것
Architecture	9	Vanilla 1개와 one-change variant 8개
Dataset	3	TIMIT, IAM Online, JSB Chorales
Random trial	각 조합 200	Variant와 dataset마다 별도 tuning
전체	27 searches, 5,400 runs	평균 24.3 CPU-hour, 합계 약 15 CPU-year

이 논문의 증거 단위는 한 최고 기록이 아니라 27개의 독립 random search다

이 논문이 답하는 것과 답하지 않는 것

Odyssey는 LSTM의 영구 순위를 정하지 않고 특정 ablation을 공정하게 비교한다

- 질문: vanilla의 구성요소 하나를 바꾸면 세 과제 성능이 어떻게 달라지는가
- 통제: 각 variant와 dataset 조합에 hyperparameter search를 따로 수행한다
- 주요 결론: 평가 범위에서 어떤 variant도 vanilla를 유의하게 능가하지 못했다
- 비결론: 모든 dataset과 optimizer에서 vanilla가 최선이라는 증명은 아니다
- 논문 목표는 state-of-the-art 기록이 아니라 architecture 구성요소 비교다

결과는 여덟 변화와 세 과제, 정해진 탐색 범위에 조건부다

1997 원형에서 Odyssey 기준 cell까지

오늘날의 vanilla LSTM은 1997 architecture에 여러 후속 개선을 더한 형태다

01	1997 original	Input·output gate, fixed CEC, full gate recurrence
↓		
02	Forget gate	Cell이 이전 state를 스스로 reset하도록 추가
↓		
03	Peephole	Cell state가 gate timing을 직접 조절
↓		
04	Full BPTT	모든 recurrent 경로의 정확한 gradient 계산
↓		
05	Odyssey	Later vanilla를 기준으로 one-change ablation

논문의 vanilla를 1997 original과 같은 cell로 부르면 forget gate의 역사를 뒤집게 된다

1997 original과 평가 대상 vanilla

두 architecture의 gate와 학습 경로가 다르므로 같은 이름 아래에서 구분해야 한다

항목	1997 original	Odyssey vanilla
Gate	Input, output	Input, forget, output
Cell self-loop	고정 CEC	Forget gate f_t 로 조절
Peephole	없음	Input·forget·output gate에 존재
Gate recurrence	Gate끼리 recurrent 연결	기본 vanilla에는 없음
Gradient	Cell 외 recurrent 경로는 truncated	Full BPTT

NFG는 later vanilla의 f_t 를 1로 고정한 ablation이지 1997 cell 전체 복원이 아니다

Vanilla cell의 여섯 계산 요소

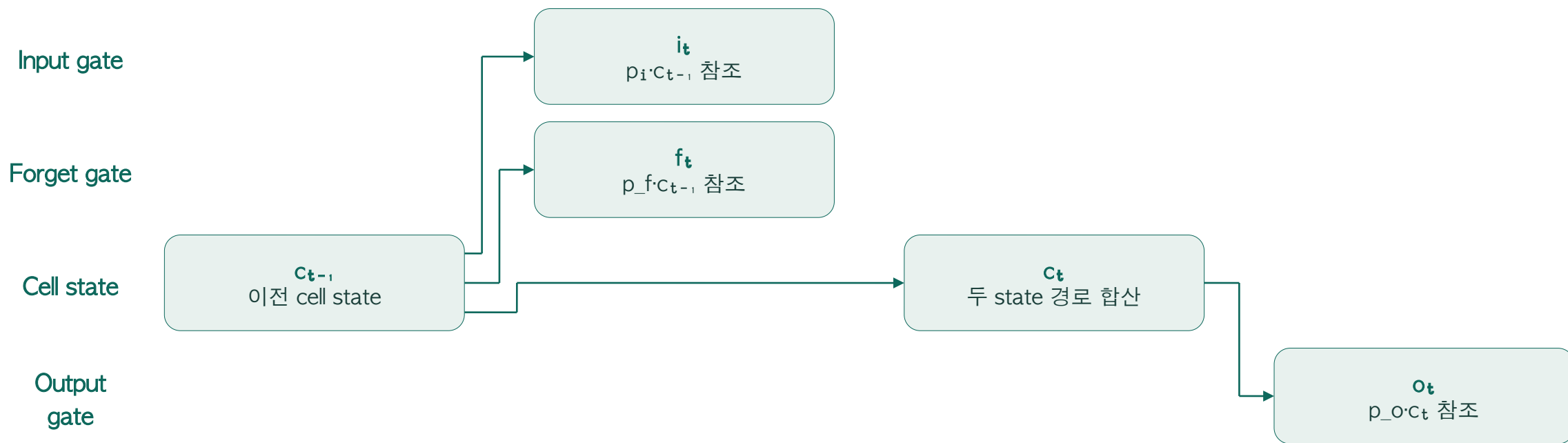
세 gate는 정보의 유입·유지·노출을 나누고 activation은 값의 범위를 조절한다

요소	기호	역할
Block input	z_t	현재 입력과 이전 출력을 새 후보 content로 변환
Input gate	i_t	새 후보 중 cell에 쓸 비율
Forget gate	f_t	이전 cell state를 유지할 비율
Output gate	o_t	활성화된 cell state를 외부에 보일 비율
Activation	g, h	논문 기본값은 둘 다 tanh

Gate는 값을 만들기보다 후보·기억·출력 흐름에 곱해지는 조절 계수다

Peephole은 cell state를 gate에 보여 준다

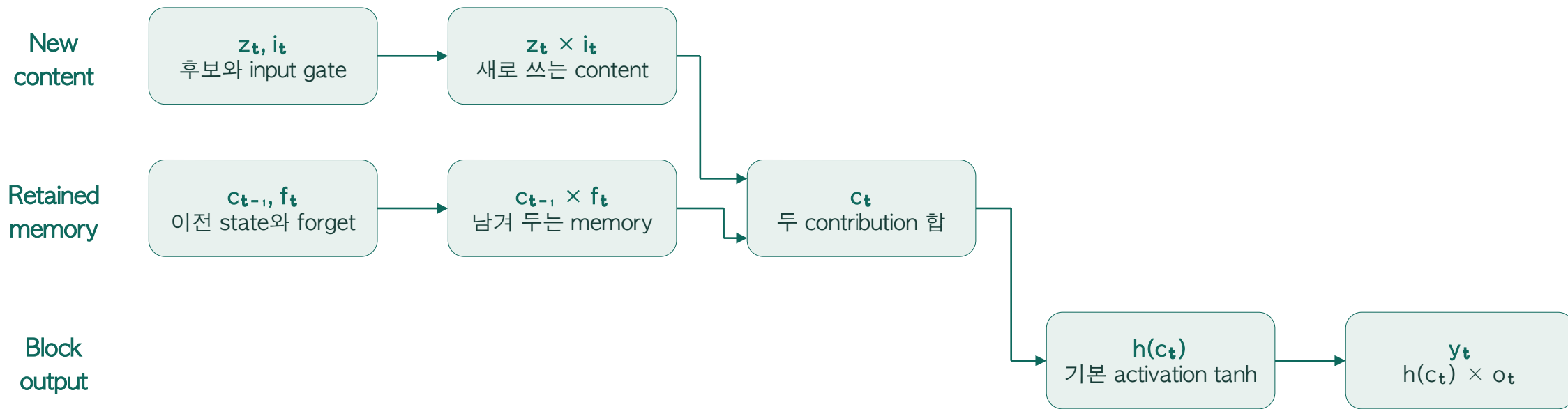
Input·forget gate는 이전 state를, output gate는 갱신된 state를 직접 참조한다



Peephole은 recurrent output y_{t-1} 과 별도로 cell state를 gate 입력에 연결한다

Cell state는 새 content와 남긴 memory의 합

두 곱셈 경로가 c_t 에서 합쳐지고 output gate가 외부 노출을 조절한다



f_t 는 이전 memory 경로에 직접 곱해져 유지와 reset의 정도를 학습한다

한 번의 scalar state update 계산

설명용 숫자로 새 content와 retained content를 분리해 계산한다

계산	대입	결과	해석
새 content	0.80×0.25	0.20	z_t 후보 중 25% 기록
남긴 memory	0.50×0.90	0.45	이전 state의 90% 유지
Cell state	$0.20 + 0.45$	$c_t = 0.65$	두 경로를 합산
Block output	$0.60 \times \tanh(0.65)$	$y_t \approx 0.343$	o_t 가 외부 노출을 조절

이 예에서 새 정보 0.20보다 남긴 memory 0.45가 c_t 에 더 크게 기여한다

Forget gate 값이 state 경로에 미치는 영향

다른 의존성을 잠시 고정하면 이전 state 경로의 local factor는 f_t 다

f_t	c_t 에 남는 비율	직관	주의
1에 가까움	이전 state 대부분 유지	긴 기억 경로	항상 보존한다는 뜻은 아님
0에 가까움	이전 state 대부분 제거	Reset에 가까움	새 content 경로는 별도
NFG	$f_t=1$ 로 고정	학습한 forgetting 없음	1997 architecture 전체와 다름

CEC의 고정 self-loop와 달리 vanilla는 f_t 로 state 유지 정도를 입력마다 바꾼다

여덟 variant는 한 번에 한 항목만 바꾼다

한 변화의 효과를 분리하려면 나머지 vanilla 구조를 기준으로 유지해야 한다

01	Vanilla V	세 gate, g와 h, peephole, 기본 recurrence
↓		
02	One change	Gate 고정, activation identity, coupling 또는 연결 추가
↓		
03	Separate tuning	Variant와 dataset 조합마다 200 trial
↓		
04	Validation	Hyperparameter와 early stopping 선택
↓		
05	Test comparison	Vanilla와 평균 차이를 통계적으로 평가

Ablation의 공정성은 같은 parameter가 아니라 각 variant를 따로 tuning하는 데 있다

Gate를 바꾸는 네 variant

약어보다 식의 literal substitution을 보면 어떤 조절 능력이 사라지는지 알 수 있다

Variant	Literal change	검사하는 질문
NIG	$i_t = 1$	새 후보를 항상 전부 쓰면 어떤가
NFG	$f_t = 1$	학습한 forgetting을 없애면 어떤가
NOG	$o_t = 1$	Cell output을 항상 전부 노출하면 어떤가
CIFG	$f_t = 1 - i_t$	쓰기와 잊기를 한 gate로 묶을 수 있는가

CIFG는 forget gate를 없애는 NFG와 달리 input gate에서 f_t 를 결정한다

Activation과 연결을 바꾸는 네 variant

각 변경은 gate 밖의 비선형성 또는 recurrent connection 하나의 역할을 분리한다

Variant	Literal change	검사하는 질문
NIAF	$g(x)=x$	Block input tanh가 필요한가
NOAF	$h(x)=x$	Cell output squash가 필요한가
NP	p_i, p_f, p_o 제거	Cell-to-gate peephole이 필요한가
FGR	Gate 간 recurrent 연결 추가	Original의 full gate recurrence가 유용한가

FGR은 gate 세 개 사이에 recurrent matrix 9개를 추가해 parameter 수를 크게 늘린다

Variant 해석에서 피해야 할 세 혼동

식 하나를 바꾼 ablation을 역사적 architecture 전체와 동일시하면 결론이 달라진다

혼동	정확한 구분	왜 중요한가
NFG = 1997 LSTM	NFG는 vanilla에서 f_t 만 1로 고정	Peephole·BPTT·recurrence 차이가 남
CIFG = no forget	f_t 를 i_t 와 결합해 계속 조절	Forget 기능은 제거되지 않음
FGR = original 복원	Gate recurrence 하나만 추가	나머지 vanilla 구성은 유지
NP = gate 제거	Cell-to-gate 항만 제거	Input·forget·output gate는 그대로

논문의 variant 이름은 전체 cell 계보가 아니라 vanilla에서 바뀐 한 항목을 가리킨다

세 dataset은 서로 다른 sequence 문제다

Metric의 단위가 다르므로 숫자를 가로로 직접 비교하지 않고 variant 내 변화로 읽는다

Dataset	문제	Metric	낮을수록
TIMIT	Audio frame의 phone 분류	Classification error %	좋음
IAM Online	Pen stroke에서 문자 sequence 인식	Character error rate	좋음
JSB Chorales	Polyphonic music next-step 예측	Negative log-likelihood	좋음

TIMIT 30%와 IAM 10%는 같은 척도가 아니며 각 dataset 안에서 비교해야 한다

과제마다 network 방향과 loss도 다르다

같은 cell variant라도 task-specific output과 training objective 안에서 평가된다

Dataset	LSTM hidden structure	Output	Training loss
TIMIT	양방향, forward와 backward layer	Softmax	Cross-entropy
IAM Online	양방향, forward와 backward layer	82-way softmax	CTC
JSB Chorales	단일 LSTM hidden layer	Sigmoid	Cross-entropy / NLL

Variant 효과는 cell만이 아니라 각 task의 model 방향과 loss 조건 안에서 관측됐다

공정 비교를 만드는 실험 pipeline

각 architecture와 dataset은 별도 탐색을 거쳐 validation 선택 후 test에서 비교된다

01 Variant × dataset 총 27개의 독립 search 문제



02 200 random trials Size, learning rate, momentum, noise를 표본화



03 Train split Full BPTT와 sequence-level SGD로 학습



04 Validation split Early stopping과 top trial 선택



05 Test split Vanilla 대비 분포와 유의성 비교

Test set은 tuning에 쓰지 않고 validation이 선택한 설정을 최종 평가한다

27개 random search는 서로 설정을 공유하지 않는다

Architecture 9개와 dataset 3개의 각 cell이 독립적인 200-trial search다

Architecture group	TIMIT	IAM	JSB
Vanilla V	200 trials	200 trials	200 trials
Gate removal 3	각 200	각 200	각 200
Activation removal 2	각 200	각 200	각 200
CIFG·NP·FGR	각 200	각 200	각 200
합계	9×200	9×200	9×200

같은 search budget을 주되 variant마다 다른 hyperparameter 조합을 찾도록 허용했다

Random search가 표본화한 범위

검색 분포까지 결과의 조건이므로 범위 밖 architecture 성능을 추론할 수 없다

Hyperparameter	Sampling	Range	의미
Hidden blocks	Log-uniform	20-200	Network size
Learning rate	Log-uniform	10^{-6} - 10^{-2}	Update 크기
Momentum	1-log-uniform	기저 0.01-1.0	Update 누적
Input noise σ	Uniform	0-1	Gaussian regularization
TIMIT only	Boolean	Momentum-clipping	추가 protocol 선택

Architecture 결론은 200회 budget뿐 아니라 이 sampling 분포와 범위에 의존한다

Training protocol의 고정 조건

Architecture 외 차이를 줄이기 위해 initialization, optimizer, 종료 규칙을 맞춘다

항목	논문 설정	해석 경계
Gradient	Full BPTT	1997 truncated training과 다름
Optimizer	Sequence별 SGD, Nesterov momentum	Modern optimizer 비교 아님
Initialization	Normal, standard deviation 0.1	모든 network에 적용
Stop	150 epochs 또는 15 epochs 개선 없음	Validation early stopping
Parameters	Variant끼리 수를 강제 동일화하지 않음	Architecture 효율 차이를 허용

Parameter 수를 같게 맞추지 않은 것은 variant마다 parameter 활용 방식이 다르기 때문이다

Figure 3 결과를 읽는 두 층

전체 200 trial과 validation-selected 상위 10%는 서로 다른 질문에 답한다

01	All 200	전체 search space에서 test 성능 분포
↓		
02	Top 10%	Validation 기준 상위 20 trial 선택
↓		
03	Test boxes	Quartile, whisker, mean, median 표시
↓		
04	Welch test	Variant 평균을 vanilla와 비교
↓		
05	Bonferroni	Variant 8개 검정의 다중 비교 보정

한 best trial보다 tuning 분포와 보정된 평균 차이가 architecture 결론의 근거다

주요 결론: 유일한 승자 variant는 없었다

평가한 여덟 변화 중 vanilla를 전반적으로 유의하게 개선한 variant는 없었다

- NFG와 NOAF는 세 dataset 모두에서 성능을 유의하게 악화시켰다
- CFIG와 NP는 평균 성능을 유의하게 낮추지 않으면서 구조를 단순화했다
- FGR은 TIMIT·IAM에서 유의한 개선이 없고 JSB에서 더 나빴다
- NIG·NOG·NIAF 효과는 task domain에 따라 달랐다
- 논문의 최종 강조는 forget gate와 output activation의 중요성이다

None of eight variants significantly improved vanilla under this evaluation

Variant 효과는 공통 결과와 task 차이로 나뉜다

세 dataset 공통 악화와 특정 domain에서만 보인 악화를 분리해 읽는다

변화	TIMIT	IAM	JSB	해석
NFG	악화	악화	악화	Forget gate 중요
NOAF	악화	악화	악화	Output squash 중요
NIG·NOG·NIAF	악화	악화	유의한 영향 없음	Domain 차이
CIFG·NP	유의 차이 없음	동일	동일	단순화 후보
FGR	유의한 개선 없음	동일	악화	추가 parameter 불리

Forget gate와 output activation의 공통 악화가 가장 일관된 architecture 증거다

CIFG와 NP가 실용적 후보로 남은 이유

유의한 개선은 아니지만 구조를 줄이면서 평가 성능을 유의하게 낮추지 않았다

Variant	줄이는 것	평균 결과	정확한 결론
CIFG	독립 forget gate parameter	세 dataset 유의 차이 없음	Input과 forget을 coupling 가능
NP	세 peephole vector	세 dataset 유의 차이 없음	Peephole 생략 가능성
공통	Parameter와 computation	Vanilla보다 유의한 향상 아님	단순화 trade-off 후보

Simpler without significant decline은 better than vanilla와 같은 문장이 아니다

Best trial 숫자는 방향 확인용이다

Validation이 고른 단일 최고 trial은 논문의 주 비교나 state-of-the-art 주장이 아니다

Dataset	Best trial	Variant	논문 해석
TIMIT	29.6% classification error	CIFG	Published best 26.9%와 별도
IAM Online	9.26% CER	NP	다른 preprocessing 결과와 직접 비교 제한
JSB Chorales	8.38 NLL	NIAF	Published 5.56보다 높은 error
용도	Rough orientation	단일 trial	Architecture 평균 결론과 분리

Best trial의 variant 이름을 전체 실험의 우승자로 바꾸면 논문의 통계 결론과 충돌한다

fANOVA는 search 결과를 중요도 지도로 바꾼다

Random trials로 성능 예측 모델을 만들고 variance를 parameter와 interaction에 배분한다

01	Random trials	Hyperparameter와 test performance 표본
↓		
02	Regression forest	100개 tree의 prediction을 평균
↓		
03	Marginalization	다른 parameter를 평균내 한 축의 효과 계산
↓		
04	Variance split	Individual과 interaction 기여로 분해
↓		
05	Importance	Search space 안의 예측 성능 변동 비율

fANOVA 중요도는 탐색 범위의 예측 성능 variance 비율이며 인과 효과가 아니다

Hyperparameter 중요도 결과

Learning rate가 가장 큰 variance를 설명했고 network size가 그다음이었다

항목	논문 관찰	조건부 해석
Learning rate	항상 variance의 2/3 초과	가장 먼저 맞출 parameter
Hidden size	두 번째로 중요	크면 좋아지나 수익 감소와 시간 증가
Input noise	TIMIT에만 중간 범위 도움	IAM·JSB에서는 대체로 해로움
Momentum	각 dataset에서 1% 미만	Online SGD 설정에서 중요도 낮음
Interactions	전체 5-20% variance	TIMIT에서 higher-order 영향 큼

Momentum의 낮은 중요도는 논문의 online SGD와 learning-rate scaling 조건에 묶인다

Parameter가 approximately independent라는 뜻

Pair interaction residual이 작고 뚜렷한 구조가 없었다는 실용적 관찰이다

관찰	가능한 행동	말하면 안 되는 결론
가장 큰 pair는 learning rate $\times$ size	Learning rate를 작은 network에서 먼저 탐색	모든 설정에서 완전 독립
Interaction map에 뚜렷한 구조 없음	Parameter를 대략 순차 tuning	Interaction이 정확히 0
전체 interaction 5-20%	Dataset별 residual 확인	Main effect만 보면 충분
IAM·JSB optimum이 범위 밖 가능	Search range를 재검토	논문 범위가 최적을 포함

Approximately independent는 탐색 절약 지침이며 수학적 독립성 증명이 아니다

논문에서 도출되는 조건부 tuning 순서

Search budget을 줄이려면 가장 큰 main effect부터 coarse-to-fine으로 확인한다

- | | | |
|----|---------------|----------------------------------|
| 01 | Small network | 빠르게 여러 learning rate를 시험 |
| ↓ | | |
| 02 | Coarse search | 높은 값부터 10배씩 낮춰 basin 확인 |
| ↓ | | |
| 03 | Choose rate | 성능이 좋고 training이 빠른 상단 선택 |
| ↓ | | |
| 04 | Increase size | Diminishing return과 시간 비용 확인 |
| ↓ | | |
| 05 | Task-specific | Noise와 interaction을 dataset별 재검증 |

이 순서는 논문의 sampled range와 online SGD에서 얻은 heuristic이다

증거의 적용 범위

대규모 실험이어도 architecture, task, optimizer, search range 밖으로 자동 일반화되지 않는다

경계	논문 범위	남는 질문
Architecture	Vanilla의 isolated change 8개	여러 변화를 결합한 cell
Task	음성·온라인 필기·polyphonic music	다른 domain과 sequence 길이
Training	Full BPTT와 online SGD	Adam류 modern recipe
Search	각 조합 200 trials, 지정 범위	더 넓은 budget과 다른 prior
시대·compute	128 AMD Opteron CPU	현대 accelerator와 scale

Forget gate 중요성은 이 ablation의 경험적 결과이며 모든 sequence model의 정리는 아니다

Odyssey에서 가져갈 네 가지

Cell 계산, 역사, 실험 설계, 통계 해석을 함께 기억해야 결론이 정확하다

축	핵심
Cell	c_t 는 $z_{t t}$ 와 $c_{t-1}f_t$ 의 합이고 y_t 는 $h(c_t)o_t$ 다
History	1997 original에는 forget gate와 peephole이 없다
Evidence	27 searches, 5,400 runs로 one-change variant 8개 비교
Result	Vanilla 개선 없음; NFG·NOAF는 세 task 모두 악화
Scope	결론은 세 task와 sampled search space에 조건부

논문의 가치는 winner 선정보다 구성요소와 tuning 직관을 통제 실험으로 검증한 데 있다

공개 출처

슬라이드의 논문 위치 표시는 공개 arXiv v2의 IEEE-layout 12쪽 판본을 기준으로 한다

출처	용도
arXiv:1503.04069	Version history와 공개 abstract
arXiv v2 PDF	수식, variant, 실험, Figure 3-6의 page locator
IEEE DOI	정식 journal publication 식별자

핵심 기술 주장과 수치는 공개 원문 page locator로 확인할 수 있다